Assessment rubric

Criteria	Weight	Novice	Developing	Competent	Proficient
OOP design and required concepts	25%	Classes are absent, mostly unused, or poorly designed; several required concepts are missing.	Some concepts are implemented, but use is superficial, inconsistent, or weakly connected to the application.	All required concepts are demonstrated in the working system with generally appropriate design.	Concepts are integrated naturally; relationships, interfaces, abstraction, and polymorphism produce a coherent, maintainable design.
Refactoring and modular architecture	15%	Phase 1 responsibilities are lost or tightly coupled; the design is monolithic or unreliable.	Some separation remains, but responsibilities overlap or dependencies are poorly managed.	Four manager responsibilities remain identifiable, with clear separation and functional interfaces.	Well-decoupled architecture with focused responsibilities, clear contracts, and a convincing refactor from Phase 1.
New features and SOLID	20%	No, poorly or not fully implemented new meaningful feature; little evidence of design principles. Little application of SOLID	One new feature is attempted but is limited, weakly integrated, or poorly justified. Some application of SOLID	One new useful feature works; at least three SOLID principles are explained with implementation evidence, including SRP OCP and LSP.	One new feature is well integrated and tested; The three applied (or more) SOLID principles meaningfully improve extensibility and maintainability.
Testing and robustness	10%	Application or tests frequently fail; little failure handling.	Basic tests exist but miss important behaviour or failure cases.	Automated tests cover core logic and the new feature, run without live AI, and relevant failures are handled.	Strong, well-chosen tests cover important edge cases; error handling and reproducibility are robust.
DevOps integration	15%	Little Git evidence; Docker or CI absent or non-functional.	Basic commits and container setup; CI is incomplete or unreliable.	Meaningful Git history, working Docker setup, and CI workflow that runs automated tests.	Clear incremental history, reliable container workflow, successful CI, and concise, reproducible documentation.
Documentation and design justification (via Engineering report) / Task Management via TeamAId (Engineering Report Page Count also taken into consideration)	10%	Required diagrams or explanations are missing or unclear. No evidence of Task Management in TeamAId.	Some documentation exists but key design decisions or evidence are unclear. Limited evidence of Task Management in TeamAId or not kept up to date. Page count over limit.	UML, data flow, exception handling, OOP mapping, and SOLID justification are clear and complete. Tasks are managed via TeamAId with clear chronological progression (not all added, completed and approved last minute!) and regular updates	Concise, accurate documentation clearly connects design choices to the implementation and domain. Tasks are very well divided in TeamAId, updated and show clear chronological progression throughout the project for each team member.
Advanced Features	5%	The advanced feature is not / not meaningfully attempted	The advanced feature is attempted but limited / weakly integrated	The advanced feature is implemented and integrated and working appropriately	The advanced feature is well integrated, tested and meaningfully adds to the purpose and function of the application

Peer Evaluation will submitted via TeamAId and be a 20% modifier of your overall project grade except in the case of zero contribution which may result in a zero grade

The rubric weights total 100% of the Phase 2 assessment; Phase 2 itself contributes 25% to the module grade.